COMPREHENSIVE TECHNICAL DOSSIER

DOCUMENT ID: CTB-ARCH-2026 • REVISION 2.5

Community Time Bank AI

Complete Technology Stack, Relational Schema, Algorithmic Engine & Feature Audit

Classification: Academic College Project Documentation • **Architecture:** Single-Origin Client + Express API + Relational Ledger • **Status:** Production Verified & Audited

1. Executive Summary & Core Philosophy

Community Time Bank AI is a full-stack, peer-to-peer service reciprocity platform designed for collegiate and local communities. The platform enables individuals to exchange practical skills, neighborly assistance, and academic tutoring using **Time Credits** rather than monetary currency.

> **The Non-Negotiable Reciprocity Principle:** One verified hour of community service equals exactly **one base Time Credit (1.0 cr)**, regardless of the conventional commercial market value of the skill. The system eliminates financial friction, empowering students and local residents to access high-value assistance through reciprocal contribution.

- **Dual-Party Atomic Confirmation:** Credits never transfer automatically upon session completion. Both the requester and helper must submit cryptographic completion confirmations in their portal before balance settlement occurs.
- **Fair Credit Multiplier System:** Standardized 1.0×, 1.25×, and 1.5× complexity multipliers accommodate tasks demanding certified expertise, physical hazards, or specialized equipment without compromising fairness.
- **Non-Monetary Community Utility:** Surplus credits can be gifted to a shared **Community Credit Pool** for hardship grants or redeemed in a **Local Perks Catalog** for event passes and cafe vouchers. Direct fiat cash withdrawals are strictly forbidden.

2. Complete Technology Stack

2.1 Frontend Client Layer

Technology	Version	Architectural Role & Implementation Details
React	`18.3.1`	Component-based UI engine utilizing functional components, custom hooks, and React Context for application state (AuthContext, ThemeContext).
TypeScript	`5.6.2`	Strict static type checking across all views, data transfer objects (DTOs), API models, and client event handlers.
Vite	`6.0.7`	High-performance development server and Rollup production bundler with reverse proxy configuration mapping `/api` to the backend on port 5001. Configured with `allowedHosts: true` for public tunneling.
Tailwind CSS	`3.4.17`	Utility-first styling with custom editorial theme configuration, dark mode support via `class` strategy, PostCSS, and Autoprefixer.
Design System	*Editorial / Bold*	Human-centered visual language: Warm Ivory (`#F7F4EC` / `#FAF8F3`), Charcoal (`#111111`), Yellow Marker (`#FFD21C`), Civic Forest Green (`#123D32`), and strict 0px border-radii.
Typography	Google Fonts	**Instrument Serif** (display headers), **DM Sans** (interface & body), and **Caveat** (editorial handwritten annotations).
Routing	`6.28.0`	`react-router-dom` with client-side history, declarative path mapping, `ProtectedRoute` session guard, and `AdminRoute` role guard.
Icons & Effects	`1.16.0`	`lucide-react` for crisp vector interface iconography; `canvas-confetti` for micro-interaction celebrations.

2.2 Backend Application & API Layer

Technology	Version	Architectural Role & Implementation Details
Node.js	`v24.x LTS`	Runtime environment operating on native ES Modules (`"type": "module"`).

Technology	Version	Architectural Role & Implementation Details
Express.js	`4.21.2`	RESTful API routing framework with JSON request parsing, URL-encoded body parsing, and centralized error handling middleware.
TypeScript Runner	`tsx 4.19.3`	Zero-compile TypeScript runtime execution with live-reloading watch mode for seamless local development.
Prisma ORM	`6.4.1`	Type-safe query builder, declarative schema management, database migrations, and ACID-compliant interactive transactions (`prisma.$transaction`).
Database Engine	SQLite / PG	Zero-friction local development powered by SQLite (`dev.db`); schema fully normalized for PostgreSQL migration.
Authentication	`jsonwebtoken 9.0.2`	Stateless Bearer JWT authentication with 7-day expiration and automated role payload extraction (`MEMBER` , `ADMIN`).
Password Hashing	`bcryptjs 3.0.2`	One-way salted hashing executed with 10 salt rounds; plaintext passwords are never persisted.
Concurrency Mgmt	`concurrently 9.1.2`	Coordinates unified root script execution for backend (port 5001) and frontend (port 5173).

3. Database Schema & Relational Architecture

The persistence layer consists of **15 relational entities** defined in `server/prisma/schema.prisma`, maintaining strict referential integrity with cascade rules and unique constraints.

Model Name	Key Attributes / Types	Purpose & Relational Cardinality
`User`	`id, email, passwordHash, role, status, creditBalance (Float)`	Core identity record. Defaults to 3.0 initial welcome credits. Relates 1:1 to Profile, 1:N to Listings, Bookings, Transactions.
`Profile`	`fullName, bio, area, phone, contactPreference, averageRating, completedCount`	Public directory details. Cascadely deleted with User. Maintains rating aggregate and completed service counters.
`Category`	`name, icon, active, multiplierEligible (Bool), maxMultiplier (Float)`	Governs skill groupings (Tech, Gardening, Home Repair, Tutoring, Creative). Dictates complexity multiplier caps.
`Skill`	`name, categoryId`	Granular skill directory (e.g., Python, Plant Pruning, Furniture Assembly).
`UserSkill`	`userId, skillId, proficiency (BEGINNER, INTERMEDIATE, EXPERT)`	Junction table mapping member skills. Composite unique index on `[userId, skillId]`.
`Availability`	`userId, weekday, startTime, endTime`	Recurring weekly calendar slots used for algorithmic availability scoring.
`Listing`	`title, description, durationHours, complexityLevel, multiplier, status`	Exchange offers and requests. Stores complexity tier (STANDARD, INTERMEDIATE, ADVANCED) and rate (1.0× to 1.5×).
`Booking`	`startAt, durationHours, agreedMultiplier, finalCreditAmount, status`	Service contracts progressing: `REQUESTED → ACCEPTED → COMPLETED / DISPUTED`.
`CompletionConfirmation`	`bookingId, userId, confirmedAt`	Two-party confirmation log. Credit transfer executes only when both requester and helper records exist.
`CreditTransaction`	`amount, baseCreditAmount, multiplier, finalCreditAmount, type, description`	Immutable append-only ledger tracking all credit transfers, signup grants, redemptions, and pool donations.
`AdminAuditLog`	`adminId, action, targetType, targetId, reason, oldValue, newValue`	Immutable administrative log recording multiplier adjustments, category policy changes, and disputes.
`Review`	`bookingId, authorId, subjectId, rating (1-5), comment`	Bilateral peer evaluation submitted after mutual booking completion.
`Report`	`reporterId, targetUserId, bookingId, reason, status, resolutionNotes`	Safety and dispute tickets reviewed by Campus Administrators.
`RedemptionOption`	`title, category, creditCost, quantityRemaining, partnerName, expiryDate`	Perks catalog (event tickets, cafe discount vouchers, donated goods).
`Redemption`	`userId, redemptionOptionId, creditsSpent, voucherCode, status`	Member voucher claims with cryptographically unique identifiers (`TB-RED-XXXX-YYYY`).
`CommunityPool`	`currentBalance, totalDonated, totalAllocated, updatedAt`	Singleton solidarity reserve tracking donated hours and audited hardship grants.

4. Core Algorithms & Mathematical Formulations

4.1 Explainable Recommendation Engine (Matching Algorithm)

When a member posts a request or searches for assistance, the recommendation engine in `matchingService.ts` evaluates all active helpers on a 100-point scale:

```
MatchScore = S_skill (40%) + S_avail (25%) + S_location (15%) + S_rating (10%) + S_reliability (10%)
```

1. Skill Relevance (Max 40 pts)
Direct skill match awards points scaled by proficiency: **Expert = 40 pts**, **Intermediate = 32 pts**, **Beginner = 24 pts**. Secondary matching skills add +4 pts each. Partial category/keyword matches receive a 20 pt baseline.

2. Availability Overlap (Max 25 pts)
Exact match on requester's preferred day awards **25 pts**. General active schedule slots award 15 pts. Unscheduled helpers receive 5 pts baseline.

3. Location Proximity (Max 15 pts)
Identical neighborhood or remote-compatible tasks award **15 pts**. Adjacent neighborhoods within the municipal area receive 7 pts.

4. Trust & Reliability (Max 20 pts)
Rating score (Max 10 pts) equals `(Rating / 5.0) × 10`. Past completion reliability (Max 10 pts) awards 10 pts for ≥ 5 completed sessions, 7 pts for 1–4 completed sessions.

4.2 Fair Credit Multiplier System

```
TransferAmount = ServiceDurationInHours × AgreedMultiplier
```

- `1.0× STANDARD` General tutoring, language exchange, plant care, conversational assistance.
- `1.25× INTERMEDIATE` Certified instruction, complex software debugging, moderate physical effort.
- `1.5× ADVANCED CAP` Component-level hardware repair, hazardous tools, emergency support. Strictly capped at 1.5×.

5. Application Features & User Routes

5.1 Public Interfaces

- `/` **(Landing Page):** Editorial manifesto, live Fair Credit Multiplier Calculator sandbox, 5 exchange principles, and interactive persona launcher.
- `/auth` **(Authentication):** Login and registration forms with a 1-click persona bar for instant evaluator switching without manual typing.
- `/terms` & `/privacy`: Complete regulatory, governance, data safety, and non-monetary compliance documentation.

5.2 Authenticated Member Workspace

- `/dashboard`: Operational control hub displaying live credit balance, completed exchange count, upcoming bookings timeline, and AI helper recommendations.
- `/find-help`: Service marketplace with multi-tier filters, category search, proximity matching, and instantaneous booking requests.
- `/my-listings`: Listing manager allowing members to create, edit, pause, and close help offers and requests with complexity tier selection.
- `/bookings`: Bilateral lifecycle manager tracking sessions from requested to completed, featuring mutual completion confirmations.
- `/wallet`: Transparent ledger tracking personal balance, total earned, total spent, filterable transaction histories, and breakdown receipts.
- `/redeem`: Perks and vouchers catalog for non-monetary credit redemptions with cryptographic voucher generation.
- `/community-pool`: Solidarity reserve enabling members to donate surplus hours with dedication messages and view public aid distributions.
- `/profile`: Personal skills manager, proficiency tag selector, contact preferences, and recurring availability calendar.
- `/messages`: In-app direct communication interface between exchange participants.

- `/community-events`: Local group activities, community gardens, repair cafes, and workshop calendar.
- `/ai-assistant`: Natural language concierge answering reciprocity policy queries and recommending helpers.

5.3 Administrative Command Center (`AdminRoute`)

- `/admin` **(Member Moderation):** Directory search, balance inspection, and account suspension/reactivation.
- `/admin` **(Policy Governance):** Category creation, multiplier eligibility toggles, and maximum multiplier ceiling configuration.
- `/admin` **(Dispute Resolution):** Review flagged bookings, inspect communication logs, and execute binding credit adjustments.
- `/admin` **(Pool Distributions):** Allocate hardship assistance from the shared community pool with mandatory immutable audit reasons.
- `/admin` **(Voucher Audit):** Partner voucher validation and inventory restocking.

6. Pre-Seeded Personas & Test Credentials

Persona	Role	Email	Password	Profile & Skill Specialization
Alice Johnson	`MEMBER`	`member.a@college.edu`	`password123`	Literature student • Requester persona • Downtown • 4.0 cr balance
Bob Smith	`MEMBER`	`member.b@college.edu`	`password123`	CS Sophomore • Expert Python, Wi-Fi & Hardware Setup • 3.0 cr balance
Carlos Mendez	`MEMBER`	`member.c@college.edu`	`password123`	Urban Gardener • Vegetable Cultivation & Math Tutor • 5.0 cr balance
Elena Rostova	`MEMBER`	`member.d@college.edu`	`password123`	Senior Designer • UI/UX, Graphic Design & Resume Review • 4.5 cr balance
Campus Admin	`ADMIN`	`admin@college.edu`	`admin123`	Platform Safety Officer • Dispute Arbitrator • 100.0 cr reserve

7. Directory Structure & Build Verification

```
TY-PROJECT-2/
├── package.json                  # Concurrently orchestrator (npm run dev)
├── client/                       # React 18 + Vite 6 + Tailwind CSS Frontend
│   ├── src/
│   │   ├── components/           # BookingModal, MatchCard, Sidebar, Navbar, DemoSwitcher
│   │   ├── context/              # AuthContext, ThemeContext
│   │   ├── pages/                # 16 routes (Dashboard, FindHelp, Bookings, Wallet, Admin, Pool)
│   │   └── services/             # API Axios/Fetch client
│   └── vite.config.ts            # Dev proxy (/api → :5001) & tunnel host permissions
└── server/                       # Node.js + Express + Prisma ORM Backend
    ├── prisma/
    │   ├── schema.prisma         # 15 Relational entities & Enums
    │   └── seed.ts               # Database reset & realistic persona seeding
    └── src/
        ├── index.ts              # Express application entry & middleware
        ├── routes/               # 12 REST controllers (auth, bookings, listings, pool, admin)
        └── services/             # matchingService.ts & ledgerService.ts
```